GÜMÜŞHANE ÜNİVERSİTESİ
İKTİSADİ ve İDARİ BİLİMLER FAKÜLTESİ
YÖNETİM BİLİŞİM SİTEMLERİ BÖLÜMÜ

YBS 207 Algoritma ve Programlamaya Giriş

List Datatype

Öğr. Gör. Dr. Baki GÖKGÖZ

Giriş

- **Temel Kavramlar ve Açıklamalar**
  - **Liste Nedir?**
    - Bir liste, bir araya gruplandırılmış öğelerin bir **koleksiyonudur**.
    - **Sıralı bir koleksiyondur**: öğelerin sabit bir konumu/indeksi vardır.
    - **Heterojen öğeler** (aynı listede farklı veri tipleri) içerebilir.
    - **Değiştirilebilirdir (mutable)**, yani öğeler değiştirilebilir, eklenebilir veya kaldırılabilir.
    - Listeler **yinelenen öğeler** (duplicate elements) içerebilir.
  - **Python'da Liste Oluşturma**
    - Köşeli parantez kullanarak:
    - Örnek: `L1 = [1, 2, 3].`
    - `list()` yapıcısını (constructor) bir yinelenebilir (iterable) nesne ile kullanarak:
      - Örnek: `L2 = list((3, 5, 7, 9))` bir demeti (tuple) listeye dönüştürür.
      - Örnek: `L3 = list('abcde')` karakterlerden oluşan bir liste oluşturur.
    - Boş bir liste oluşturma:

```
L4 = []
```

- **veya**

```
L4 = list().
```

Giriş

- **Temel Kavramlar ve Açıklamalar**
  - **Heterojen Yapı (Heterogeneous Nature)**
    - Listeler, tamsayılar, ondalık sayılar, karakter dizileri, mantıksal değerler ve karmaşık sayılar dahil birden fazla türde öğe barındırabilir.
    - Örnek: `L1 = [7, 3.2, "John", True, 5+6j]`
    - Dahili olarak, listeler nesnelere referanslar (gösterimler) saklar; bu referanslar tek tip olduğu için listeler herhangi bir nesne türünü tutabilir.
  - **Bellekte Temsil (Representation in Memory)**
    - Listeler, gerçek nesnelere işaret eden referanslar dizisi (array of references) olarak uygulanır.
    - Elemanların referansları bitişik (**contiguously**) olarak saklanır, ancak her bir elemanın referansı ayrı bir bellek konumundaki nesneyi işaret eder.
    - İndeksleme 0'dan başlar (pozitif indeksler) ve ayrıca -1'den başlayan negatif indeksler (son eleman) kullanılarak da erişilebilir.

Giriş

- **Temel Kavramlar ve Açıklamalar**
  - **Değiştirilebilirlik (Mutability)**
    - Belirli bir indeksteki elemanları değiştirebilirsiniz.
    - .append() gibi metotlar kullanarak eleman ekleyebilirsiniz.
    - .remove() gibi metotlar kullanarak eleman kaldırabilirsiniz.
    - Örnek: Bir elemanın değerini atama yoluyla değiştirme: `L1[2] = 15`
  - **Listelerde Gezinme (Traversal)**
    - Listeler, for döngüsü gibi döngüler kullanılarak gezinilebilir/taranabilir (traversed).
    - Örnek: `for x in L1: print(x)` ifadesi, listedeki her bir elemanı yazdırır.

Giriş

- **Temel Kavramlar ve Açıklamalar**
  - **Örnekler**
    - **Köşeli Parantez Kullanarak Liste Oluşturma**

```
L1 = [1, 2, 3, 4, 5, 6]
print(L1) # Output: [1, 2, 3, 4, 5, 6]
```

    - Python'da köşeli parantez [] kullanarak bir tamsayı listesi oluşturmak en doğrudan ve yaygın yöntemdir.
    - **`list()` kullanarak Demetten (Tuple) Liste Oluşturma**

```
L2 = list((3, 5, 7, 9))
print(L2) # Output: [3, 5, 7, 9]
```

    - **`list()` Kullanarak Bir Karakter Dizisinden (String) Liste Oluşturma**

```
L3 = list('abcde')
print(L3) # Output: ['a', 'b', 'c', 'd', 'e']
```

    - **`Heterojen Liste Örneği`**

```
L1 = [7, 3.2, 'John', True, 5+6j]
print(L1)
```

Giriş

- **Temel Kavramlar ve Açıklamalar**
  - **Örnekler**
    - **Pozitif ve Negatif İndeksler Kullanarak Elemanlara Erişim**

```
print(L1[2]) # Output: 'John'
print(L1[-1]) # Output: (5+6j)
```

    - **Köşeli Parantez Kullanarak Liste Oluşturma**

```
L1 = [1, 2, 3, 4, 5, 6]
print(L1) # Output: [1, 2, 3, 4, 5, 6]
```

    - **For Döngüsü Kullanarak Liste Elemanlarında Gezinme**

```
for x in L1:
    print(x)
```

    - **Bir Elemanı Değiştirme**

```
L1 = [2, 4, 6, 8, 10, 12]
L1[2] = 15
print(L1) # Output: [2, 4, 15, 8, 10, 12]
```

    - **append() Kullanarak Eleman Ekleme**

```
L1.append(25)
print(L1) # Output: [2, 4, 15, 8, 10, 12, 25]
```

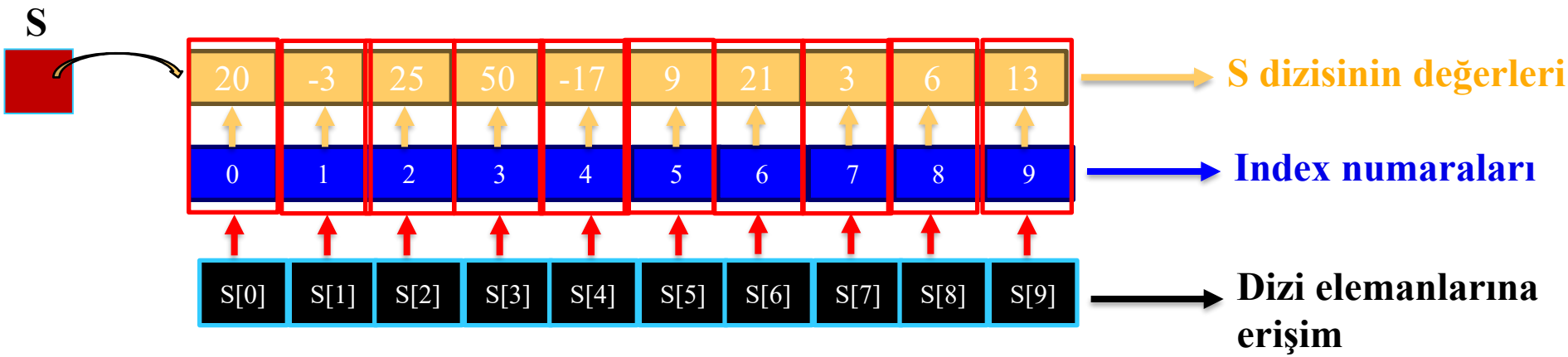
Giriş

- **Temel Kavramlar ve Açıklamalar**
  - **Örnekler**
    - **remove() Kullanarak Eleman Kaldırma**

```
L1.remove(10)
print(L1) # Output: [2, 4, 15, 8, 12, 25]
```

Indexing & Slicing

- **Indexleme**
  - İçerik olarak, bir **list'te** her bir eleman bir konum (indeks) kapladığı bir dizi olarak saklanır.
  - İndeksler ilk karakter için **0'**dan başlar ve sonraki her bir karakter için **1** artar
  - Elemanlara erişim bu indeksler kullanılarak yapılır,

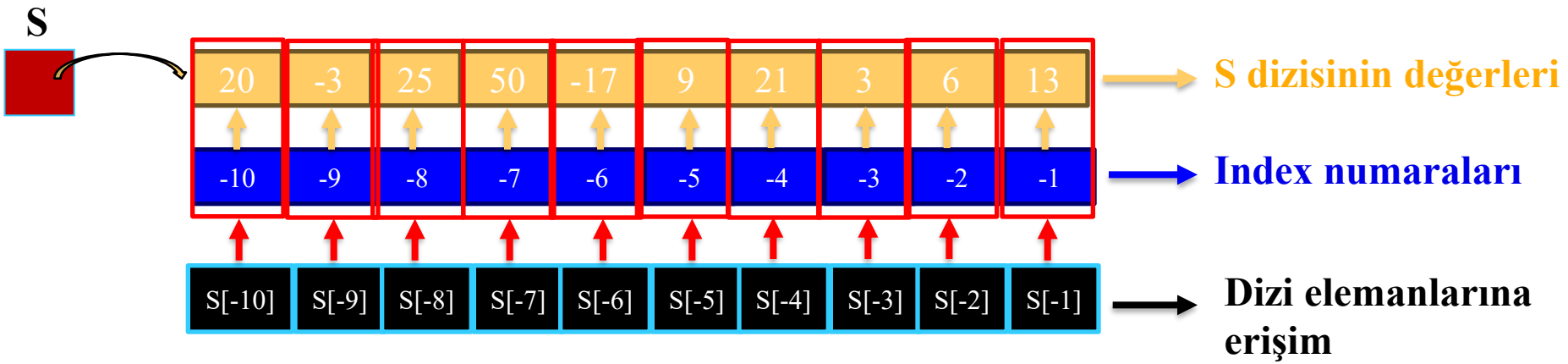



```
print(S[0])
```
→ 20

```
print(S[5])
```
→ 9

Indexing & Slicing

- **Indexleme**
  - Python ayrıca negatif indeksleri de destekler; son eleman -1'den başlayarak geriye doğru gider (örneğin, `R=-1, A=-2, ...`) .



  - Liste elemanlarına erişim bu indeksler kullanılarak yapılır, örneğin **`S[0]`** veya **S[-1]** sırasıyla ilk ve son karakteri döndürür.

```
print(S[-1])
```
→ 13

```
print(S[-4])
```
→ 21

List Uzunluğunu Ölçme

- **List Uzunluğunu Ölçme**
  - **len()** fonksiyonu, herhangi bir dizinin uzunluğunu döndürür.
  - **Örnek:** len(L1) 9 değerini döndürür.

```
L1 = [20, -3, 25, 50, -17, 9, 21, 3, 6, 13]
for x in L1:
    print(x)
```

```
L1 = [20, -3, 25, 50, -17, 9, 21, 3, 6, 13]
for i in range(len(L1)):
    print(L1[i])
```

List Slicing

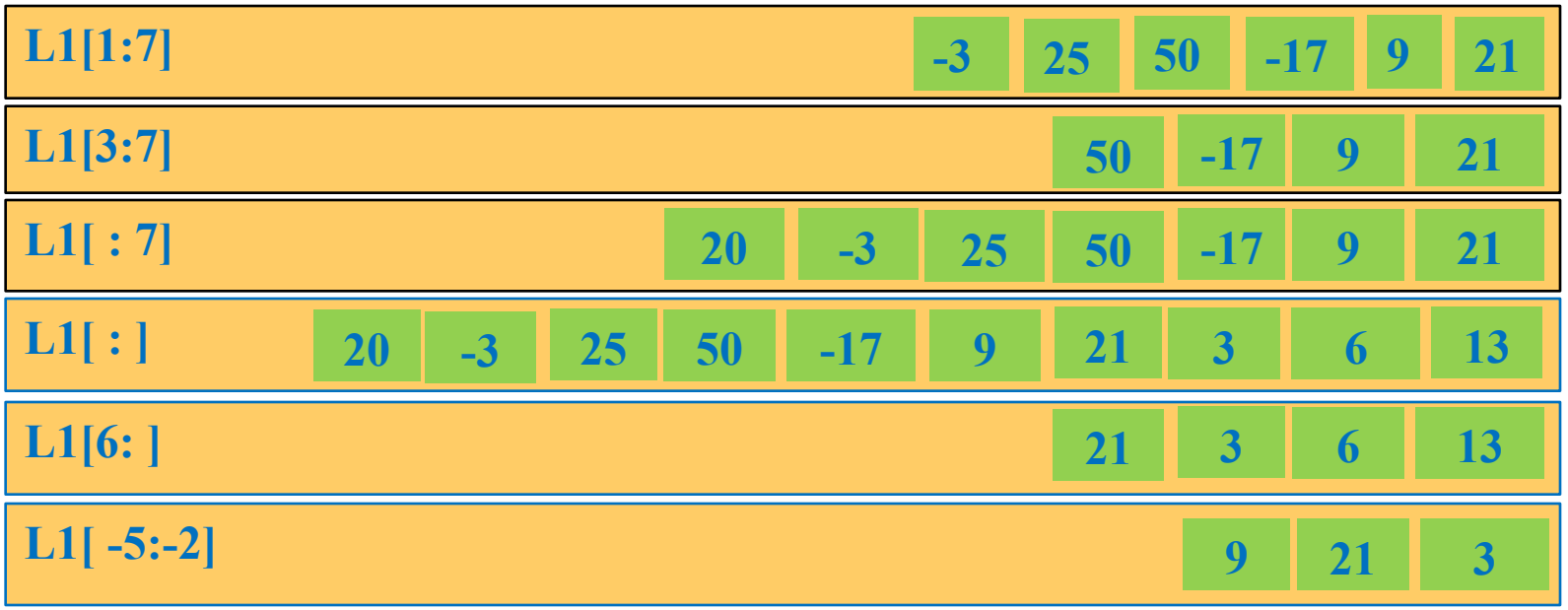
- **List Slicing Syntax:**

range [start: stop: step]

– Parametreler:
  - start: İsteğe bağlı, varsayılan = 0.
  - stop: Zorunlu, döngü bu değere kadar çalışır ancak bu değeri hariç tutar.
  - step: İsteğe bağlı, varsayılan = 1, yani artış değeri.

– Python kodları:

```
L1 = [20, -3, 25, 50, -17, 9, 21, 3, 6, 13]
```



Liste Yazma/Değiştirme İçin İndeksleme

- **List Yazma/Değiştirme İçin İndeksleme**
  - İndeksleme, belirtilen konumdaki tek bir öğeye erişir.
  - Sözdizimi: `list[indeks] = değer`
  - Pozitif veya negatif indeksler kullanılabilir (örneğin, son öğe için `-1`).
  - Örnek:

```
L1 = [1, 2, 3, 4, 5]
L1[3] = 10 # L1 listesinde 3. indexteki 4 değerini 10 ile değiştirir.
```

  - Yeni liste:

```
L1 = [1, 2, 3, 10, 5]
```

  - Örnek:

```
L1 = [1, 2, 3, 4, 5]
L1[3] = [10, 11]
print(L1)
# output [1, 2, 3, [10, 11], 5]
```

Liste Yazma/Değiştirme İçin İndeksleme

- **Liste Yazma/Değiştirme için Dilimleme**
  - Dilimleme sözdizini:

List [start: stop: step]

  - Birden fazla öğeyi değiştirmek veya yeni öğeler eklemek için kullanılır.

```
L1 = [10, 20, 30, 40, 50]
```

  - Yalnızca başlangıç ve bitiş belirtildiğinde:
    - Değiştirilecek öğe sayısıyla sınırlı olmaksızın, istenilen sayıda öğe atanabilir.

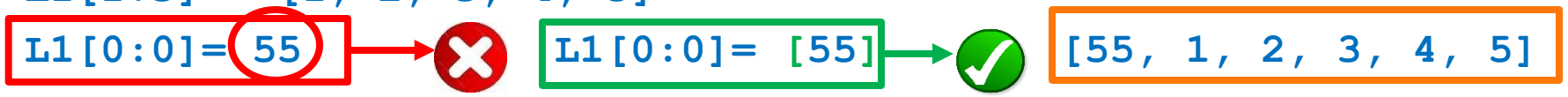
List [start : stop]

Örnek:

```
L1 = [10, 20, 30, 40, 50]
L1[1:3] = [1, 2, 3, 4, 5]
print(L1)  #output:  [10, 1, 2, 3, 4, 5, 40, 50]
```

Örnek:

```
L1[1:3] = [1, 2, 3, 4, 5]
```



Liste Yazma/Değiştirme İçin İndeksleme

- **Liste Yazma/Değiştirme için Dilimleme**
  - Aralık dışı öğe ekleme
  - **Örnek:**

```
L1 = [10, 20, 30, 40, 50]
L1[9:9] = [10]
print(L1)# output [10, 20, 30, 40, 50, 10]
```

  - Birden fazla öğenin daha az veya daha fazla öğeyle değiştirilmesi
  - **Örnek1:**

```
L1 = [1, 2, 3, 4, 5]
L1[1:4] = [10]
print(L1) # output [1, 10, 5]
```

  - **Örnek2:**

```
L1 = [1, 2, 3, 4, 5]
L1[1:4] = [10, 11]
print(L1) # [1, 10, 11, 5]
```

  - **Örnek3**

```
L1 = [1, 2, 3, 4, 5]
L1[1:4] = [10, 11, 12, 13]
print(L1) # [1, 10, 11, 12, 13, 5]
```

Liste Birleştirme ve Tekrarlama

- **Liste Yazma/Değiştirme için Dilimleme**
  - Öğeleri adımla değiştirin - gereken tam değiştirme sayısı
  - **Örnek:**

```
L1 = [1, 2, 3, 4, 5]
L1[::2] = [10, 11, 12]
print(L1)
# output [10, 2, 11, 4, 12]
```

  - Negatif adımla dilimleme ataması yoluyla liste öğelerini tersine çevirme
  - **Örnek:**

```
L1 = [1, 2, 3, 4, 5]
L1[::-1] = [10, 11, 12, 13, 14]
print(L1)
# output [14, 13, 12, 11, 10]
```

  - Negatif adımla ters kısmi dilimleme

Örnek:

```
L1 = [1, 2, 3, 4, 5]
L1[3:0:-1] = [12, 13, 14]
print(L1)
# output [1, 14, 13, 12, 5]
```

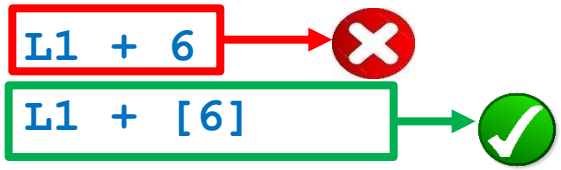
Liste Birleştirme ve Tekrarlama

- **Birleştirme (`+` operatörü ve `extend()` metodu)**
  - `+` kullanılarak iki listeyi orijinalleri değiştirilmeden yeni bir listede birleştirilebilir
  - **Örnek:**

```
L1 = [1, 2, 3]
L2 = [8, 9, 10]
L3 = L1 + L2
print(L3)
# Output: [1, 2, 3, 8, 9, 10]
```

  - Bir listeyi liste dışı bir türle (örneğin, tam sayı) birleştirmeye çalışmak `TypeError` hatasına neden olur.
  - **Örnek:**

```
L1 = [1, 2, 3, 4, 5]
L1 + 6
L1 + [6]
```



Liste Birleştirme ve Tekrarlama

- **Birleştirme (`+` operatörü ve `extend()` metodu)**
  - **extend()** fonksiyonu kullanılarak bir listenin elemanlarını diğerine ekleyebilir ve liste değiştirilebilir.

Örnek:

```
L1 = [1, 2, 3, 4, 5]
L1.extend([4, 5, 6])
print(L1)
# Output: [1, 2, 3, 4, 5, 4, 5, 6]
```

Liste Birleştirme ve Tekrarlama

- **Tekrarlama (`*`) operatörü**
  - `*` Bir listeyi bir tam sayı ile çarparak, öğelerini tekrarlayarak yeni bir liste oluşturur.
  - **Örnek:**

```
L1 = [1, 2, 3]
L2 = L1*2
print(L2) # Output: [1, 2, 3, 1, 2, 3]
```

  - Tekrarlama sayısı bir tam sayı olmalıdır; kayan noktalı sayılar `TypeError` hatasına neden olur.
  - Bir listeyi liste dışı bir türle (örneğin, tam sayı) birleştirmeye çalışmak **`TypeError`**

Liste Birleştirme ve Tekrarlama

- **Listede Bulunma İşlemleri (`in`, `not in`)**
  - Bir öğenin bir listede olup olmadığını kontrol eder.
  - İç içe geçmiş listelerde, bir alt listenin öğe olarak var olup olmadığını kontrol eder.
  - Genellikle liste öğeleri üzerinde yineleme yapmak için `for` döngülerinde kullanılır.
  - **Örnek:**

```
L1 = [3, 4, 5]
print(3 in L1) # Output: True
print(7 in L1) # Output: False
```